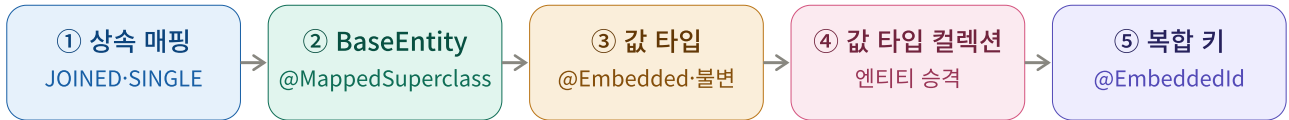


[Lv2] JPA 4주차 학습 노트

고급 매핑과 객체지향 모델링: 상속, 값 타입, 복합 키 정보

객체의 **상속**을 DB에 저장하는 정석과 꿈수, 모든 실무 프로젝트의 0순위 필수템 **BaseEntity**, "주소는 객체로 묶고 싶다"는 욕망을 실현하는 임베디드 타입, 값 타입 컬렉션의 '전체 삭제 후 재저장' 참사 방어, 레거시 DB 연동의 관문인 복합 키까지 — 데이터 모델링의 밑바닥부터 심화까지.

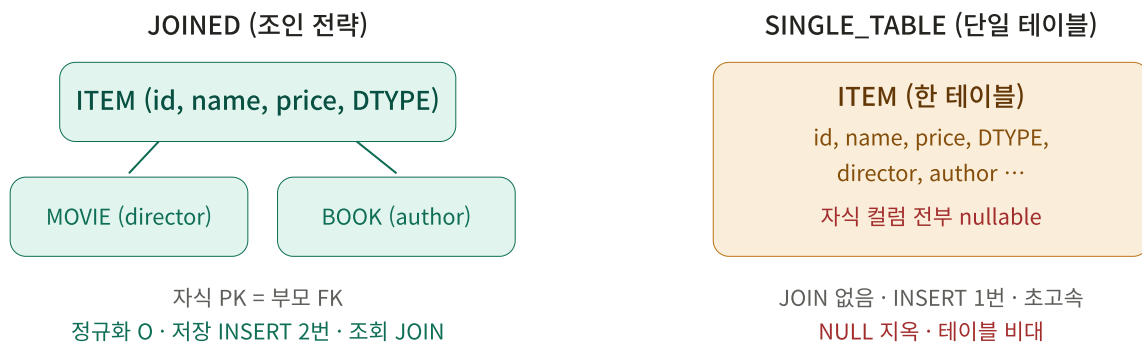


객체지향 설계 기법을 DB에 적용해 재사용성과 응집도를 극대화한다

▲ 4주차 전체 흐름

① 상속관계 매핑 — DB엔 상속이 없다

자바엔 **Item** (부모) ← **Movie**, **Book** 이 있지만 RDB엔 '상속' 개념이 없다. 대신 **슈퍼타입-서브타입** 모델링으로 매핑하며, 두 가지 전략이 있다.



▲ 조인 전략(정규화·정석) vs 단일 테이블 전략(성능·NULL 허용)

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED) // 전략만 바꾸면 구조가 바뀐
@DiscriminatorColumn(name = "DTYPE") // 자식 타입 구분 컬럼 (실무 권장)
public abstract class Item { // 부모는 abstract 권장
    @Id @GeneratedValue @Column(name = "ITEM_ID")
    private Long id;
    private String name;
```

```

    private int price;
}

@Entity @DiscriminatorValue("M")    // DTYPE 컬럼에 'M' 저장
public class Movie extends Item {
    private String director;
    private String actor;
}

```

비교	JOINED	SINGLE_TABLE
구조	정규화(깔끔)	비정규화(똥똥)
성능	JOIN 발생	매우 빠름 (단순 SELECT)
저장공간	효율적	비효율(NULL 많음)
추천	[기본] 복잡·중요할 때	단순하거나 조회성능 중요할 때

선택 가이드 기본은 **JOINED**(객체지향·정규화). 비즈니스가 정말 단순하고 확장 가능성이 낮으면 **SINGLE_TABLE**. 두 전략 모두 **DTYPE**은 넣는 걸 권장(**SINGLE_TABLE**은 필수).

② @MappedSuperclass — BaseEntity로 공통 필드 상속

"모든 테이블에 등록일·수정일을 넣어주세요" 요구에, 엔티티 100개를 복붙하지 않고 **공통 필드(Auditing)**를 한 곳에서 관리하는 실무 필수 패턴. 부모는 **테이블이 되지 않고** 매핑 정보(필드)만 자식에 복사된다.

```

@MappedSuperclass                                // "난 엔티티가 아냐. 매핑 정보만 빌려줄게"
@EntityListeners(AuditingEntityListener.class) // 날짜 자동 기입
public abstract class BaseEntity {
    @CreatedDate @Column(updatable = false)    // 생성일은 절대 수정 안 됨
    private LocalDateTime createdDate;
    @LastModifiedDate
    private LocalDateTime lastModifiedDate;
}
// 메인 클래스에 @EnableJpaAuditing 필수! 자식은 extends BaseEntity만 하면 끝

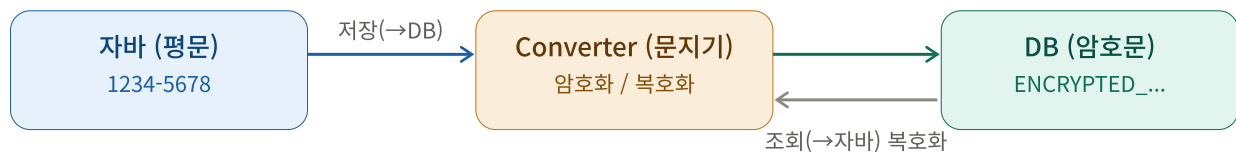
```

비교	@Inheritance	@MappedSuperclass
본질	Is-A ("영화는 상품이다")	공통 속성 공유 (Mixin)
부모 테이블	됨 (@Entity)	안 됨 (필드만 복사)
다형성 쿼리	가능 (부모 타입 조회)	불가능 (매핑 테이블 없음)
주 사용처	비즈니스 계층구조	시스템 공통 필드(등록일 등)

✅ **실무 빈도** 상속관계 매핑(@Inheritance)은 복잡도가 높아 꼭 필요할 때만 쓰고, **BaseEntity(@MappedSuperclass)**는 거의 **100%** 사용한다. 컬럼명을 바꾸려면 자식에서 **@AttributeOverride**로 재정의.

실무 결제 도메인: 상속 + AttributeConverter

결제(Payment) 부모 ← 카드결제(CardPayment) 자식을 **JOINED**로 매핑하고, 카드번호는 **AttributeConverter**로 자동 암호화한다.



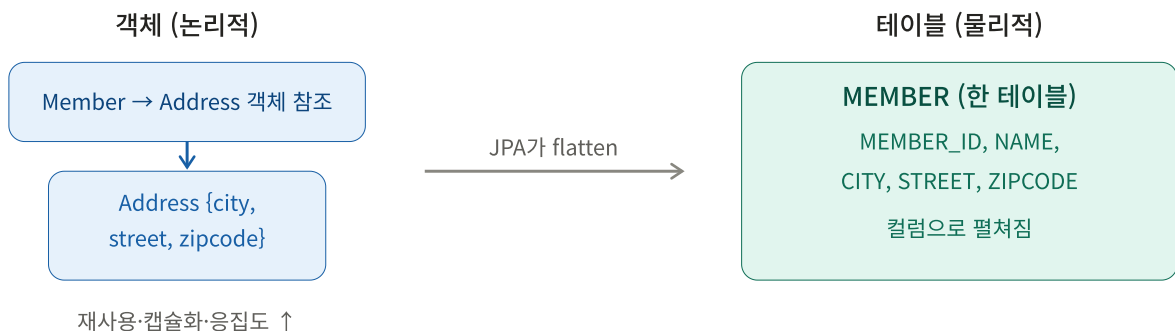
▲ AttributeConverter: 자바↔DB 사이의 통역사 (암호화-Y/N-레거시 코드 변환)

```

@Converter
public class CryptoConverter implements AttributeConverter<String, String> {
    public String convertToDatabaseColumn(String attr) {    // 자바→DB (암호화)
        return attr == null ? null : "ENCRYPTED_" + attr;
    }
    public String convertToEntityAttribute(String db) {     // DB→자바 (복호화)
        return db == null ? null : db.replace("ENCRYPTED_", "");
    }
}
// 적용: @Convert(converter = CryptoConverter.class) private String cardNumber;
// @Converter(autoApply = true) 로 특정 타입 전체에 글로벌 적용도 가능
  
```

③ 값 타입 (1) 임베디드 타입 — 응집도 높은 모델링

회원엔 `city`, `street`, `zipcode` 가 흩어져 있으면 가독성·응집도·재사용성이 나쁘다. 이를 `Address` 하나로 묶는다 (`@Embeddable` / `@Embedded`). 핵심은 객체는 잘게 쪼개지만(Fine-Grained), 테이블은 그대로 컬럼으로 퍼진다(Flatten).



▲ 임피던스 불일치의 타협 — 개발자는 객체로, DB는 한 테이블로

```

@Embeddable
public class Address {
    private String city, street, zipcode;
    public Address() {} // JPA 스펙상 기본 생성자 필수
    public Address(String city, String street, String zipcode) { ... }
}

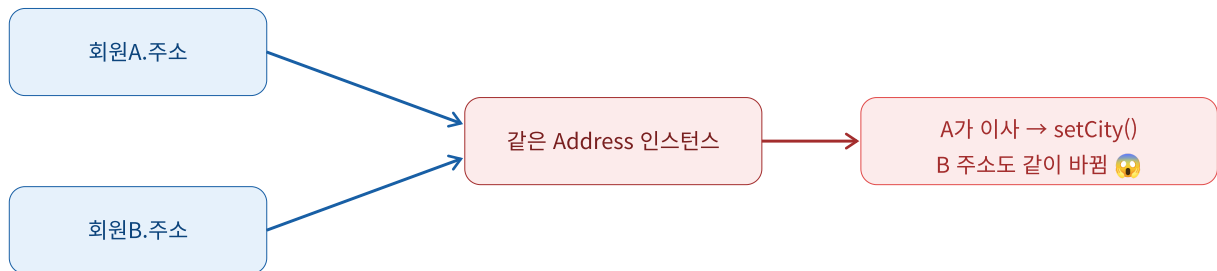
@Entity
public class Member {
  
```

```
@Embedded private Address homeAddress; // 이 필드는 값 타입을 내장
}
```

같은 타입을 두 번 쓸 때 한 엔티티에 Address 를 home/work 두 번 쓰면 컬럼명 충돌(MappingException).
@AttributeOverrides 로 WORK_CITY 처럼 재정의한다.

④ 값 타입 (2) 불변 객체와 비교

임베디드 타입은 자바 객체라 대입(=) 시 참조가 공유된다. 그래서 공유된 값 타입 하나를 고치면 다른 곳도 같이 바뀌는 **사이드 이펙트**가 터진다.



▲ 공유 참조 사이드 이펙트 — 에러 로그도 안 남아 찾기 힘든 버그

✅ **해결: 불변 객체(Immutable Object) Setter를 아예 만들지 않는다.** 생성자로만 값을 세팅하고, 수정은 새 객체를 만들어 통째로 교체 (replace)한다. 그러면 공유해도 값을 못 바꿔 부작용이 원천 차단된다.

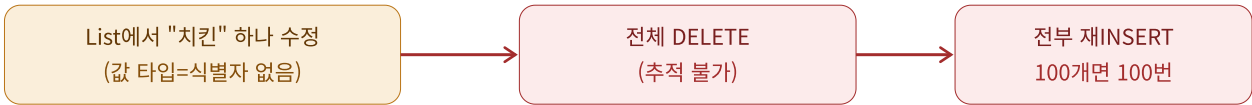
엔티티 vs 값 타입 & equals 재정의

구분	엔티티	값 타입
식별자	있음 (@Id)	없음
생명주기	독립적	의존적(엔티티 죽으면 같이)
비교 기준	PK(ID)가 같으면 같음	모든 필드 값이 같으면 같음

값 타입은 "내용이 같으면 같은 것"이므로 == (주소 비교)가 아니라 equals() (값 비교)를 써야 한다. 따라서 equals() 와 hashCode() 를 반드시 재정의(Lombok @EqualsAndHashCode)해야 HashSet · HashMap 에서 정상 동작한다.

⑤ 값 타입 컬렉션 — 함정과 대안

"좋아하는 음식 리스트"처럼 값을 여러 개 담으려면 @ElementCollection + @CollectionTable 로 별도 테이블에 저장한다. 하지만 치명적 함정이 있다.



해결: 일대다(1:N) 엔티티로 승격 — 식별자(PK) 부여
 @OneToMany + cascade=ALL + orphanRemoval=true (양방향 mappedBy로 불필요 UPDATE 제거)

▲ "전체 삭제 후 재저장" 참조와 엔티티 승격 해결책

⚠ **왜 이런 일이?** 값 타입은 식별자가 없어 DB가 "어느 게 바뀐 건지" 추적할 수 없다. 그래서 List 수정 시 해당 회원의 데이터를 전부 지우고 다시 넣는다. 100개면 하나 고치려고 100번 재저장.

✅ **@ElementCollection은 언제?** 추적 불필요·소량·불변 데이터에만(체크박스 다중선택 등). "주소 이력"처럼 쌓이거나 변경이 잦으면 무조건 엔티티로 승격. 단방향(@JoinColumn)이면 불필요 UPDATE가 생기므로 양방향(mappedBy + 자식이 @ManyToOne으로 FK 관리)이 깔끔하다.

⑥ 복합 키 — @IdClass vs @EmbeddedId

"주문번호+상품번호"처럼 컬럼 2개 이상을 묶어 PK로 쓸 때, 식별자 클래스는 **Serializable** + **equals** / **hashCode** + 기본 생성자 + **public**이 필수다.

특징	@IdClass	@EmbeddedId
스타일	DB 친화적	객체 친화적
식별자 클래스	POJO 가능	@Embeddable 필수
엔티티 필드	펼쳐짐(id1, id2)	뭉쳐짐(id)
조회	<code>entity.getId1()</code>	<code>entity.getId().getId1()</code>

```

// @EmbeddedId (실무 선호)
@Embeddable
public class SlotId implements Serializable { // Serializable 필수
    private String charId; private int slotNum;
    // @EqualsAndHashCode, 기본생성자 필수
}
@Entity
public class ItemSlot {
    @EmbeddedId private SlotId id; // ID 객체를 통째로 박아넣음
    private String itemName;
}
  
```

식별 관계와 @MapsId

자식이 부모 PK를 받아 자신의 PK이자 FK로 동시에 쓰는 구조(식별 관계). **@MapsId** 는 연관관계로 받은 부모의 ID를 자식 복합 키에 자동 복사해준다.

```
@Entity
public class Comment {
    @EmbeddedId private CommentId id;

    @MapsId("postId")           // post의 ID를 CommentId.postId에 자동 복사
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "POST_ID")
    private Post post;
}
// comment.setPost(post)만 하면 postId를 수동으로 안 넣어도 됨
```

💡 **실무 추천: 대리 키(Surrogate Key)** 복합 키는 복잡하다. 신규 프로젝트라면 그냥 **Long id (@GeneratedValue)** 하나를 PK로 쓰고, 복합 조건은 **유니크 제약조건**으로 거는 게 정답. **@MapsId** ·식별 관계는 어쩔 수 없이 **레거시 DB를 매핑할 때만** 쓰는 구조용 밧줄.

4주차 한 문장 요약

객체의 상속은 **JOINED(정석)/SINGLE_TABLE(성능)**로 매핑하고, 공통 필드는 **BaseEntity(@MappedSuperclass)**로 뽑는다. 흩어진 필드는 **임베디드 값 타입**으로 묶되 반드시 **불변 객체**로 만들어 사이드 이펙트를 막고, 값 타입 컬렉션은 함정이 많으니 **엔티티로 승격**한다. 복합 키는 **@EmbeddedId**가 표준이지만 신규 설계는 **Long 대리 키**가 정답.

셀프 체크 (복습용)

- Q1.** 상속 매핑에서 기본으로 뭘 쓰고, 언제 단일 테이블을 쓰나?
A. 기본 JOINED(정규화). 단순하거나 조회 성능이 매우 중요할 때만 SINGLE_TABLE(자식 컬럼 전부 nullable 감수).
- Q2.** @Inheritance와 @MappedSuperclass의 결정적 차이는?
A. 부모 타입 다형성 쿼리 가능(@Inheritance) vs 불가(@MappedSuperclass는 매핑 테이블 없음).
- Q3.** 값 타입을 왜 불변으로 만드나?
A. 참조 공유 시 한쪽을 고치면 다른 쪽도 바뀌는 사이드 이펙트를 막기 위해. Setter를 없애고 새 객체로 교체.
- Q4.** 값 타입 컬렉션의 치명적 단점과 해결책은?
A. 식별자가 없어 수정 시 전체 삭제 후 재저장. 일대다 엔티티로 승격(PK 부여).
- Q5.** 신규 프로젝트에서 복합 키 대신 뭘 쓰나?
A. Long 대리 키 + 유니크 제약조건. @EmbeddedId/@MapsId는 레거시 매핑용.